Similarity Index	Formula	Invariant to: Invertible Linear Transform	Orthogonal Transform	Isotropic Scaling
Linear Reg. (R^2_{LR})	$\|\|Q_Y^{\text{T}}X\|\|_{\text{F}}^2/\|\|X\|\|_{\text{F}}^2$	Y only	✓	✓
CCA (R^2_{CCA})	$\|\|Q_Y^{\text{T}}Q_X\|\|_{\text{F}}^2/p_1$	✓	✓	✓
CCA ($\bar{\rho}_{\text{CCA}}$)	$\|\|Q_Y^{\text{T}}Q_X\|\|_*/p_1$	✓	✓	✓
SVCCA (R^2_{SVCCA})	$\|\|(U_YT_Y)^{\text{T}}U_XT_X\|\|_{\text{F}}^2/\min(\|\|T_X\|\|_{\text{F}}^2, \|\|T_Y\|\|_{\text{F}}^2)$	If same subspace kept	✓	✓
SVCCA ($\bar{\rho}_{\text{SVCCA}}$)	$\|\|(U_YT_Y)^{\text{T}}U_XT_X\|\|_*/\min(\|\|T_X\|\|_{\text{F}}^2, \|\|T_Y\|\|_{\text{F}}^2)$	If same subspace kept	✓	✓
PWCCA	$\sum_{i=1}^{p_1} \alpha_i\rho_i/\|\|\alpha\|\|_1$, $\alpha_i = \sum_j \|\langle \mathbf{h}_i, \mathbf{x}_j\rangle\|$	✗	✗	✓
Linear HSIC	$\|\|Y^{\text{T}}X\|\|_{\text{F}}^2/(n-1)^2$	✗	✓	✗
Linear CKA	$\|\|Y^{\text{T}}X\|\|_{\text{F}}^2/(\|\|X^{\text{T}}X\|\|_{\text{F}}\|\|Y^{\text{T}}Y\|\|_{\text{F}})$	✗	✓	✓
RBF CKA	$\text{tr}(KHLH)/\sqrt{\text{tr}(KHKH)\text{tr}(LHLH)}$	✗	✓	✓*

Table 1. Summary of similarity methods investigated. Q_X and Q_Y are orthonormal bases for the columns of X and Y. U_X and U_Y are the left-singular vectors of X and Y sorted in descending order according to the corresponding singular vectors. $||\cdot||_*$ denotes the nuclear norm. T_X and T_Y are truncated identity matrices that select left-singular vectors such that the cumulative variance explained reaches some threshold. For RBF CKA, K and L are kernel matrices constructed by evaluating the RBF kernel between the examples as in Section 3, and H is the centering matrix $H_n = I_n - \frac{1}{n}\mathbf{1}\mathbf{1}^{\text{T}}$. See Appendix C for more detail about each technique.
*Invariance of RBF CKA to isotropic scaling depends on the procedure used to select the RBF kernel bandwidth parameter. In our experiments, we selected the bandwidth as a fraction of the median distance, which ensures that the similarity index is invariant to isotropic scaling.

For a linear kernel, CKA is equivalent to the RV coefficient (Robert & Escoufier, 1976) and to Tucker's congruence coefficient (Tucker, 1951; Lorenzo-Seva & Ten Berge, 2006).

Kernel Selection. Below, we report results of CKA with a linear kernel and the RBF kernel $k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-||\mathbf{x}_i - \mathbf{x}_j||_2^2/(2\sigma^2))$. For the RBF kernel, there are several possible strategies for selecting the bandwidth σ, which controls the extent to which similarity of small distances is emphasized over large distances. We set σ as a fraction of the median distance between examples. In practice, we find that RBF and linear kernels give similar results across most experiments, so we use linear CKA unless otherwise specified. Our framework extends to any valid kernel, including kernels equivalent to neural networks (Lee et al., 2018; Jacot et al., 2018; Garriga-Alonso et al., 2019; Novak et al., 2019).

4. Related Similarity Indexes

In this section, we briefly review linear regression, canonical correlation, and other related methods in the context of measuring similarity between neural network representations. We let Q_X and Q_Y represent any orthonormal bases for the columns of X and Y, *i.e.* $Q_X = X(X^{\text{T}}X)^{-1/2}$, $Q_Y = Y(Y^{\text{T}}Y)^{-1/2}$ or orthogonal transformations thereof. Table 1 summarizes the formulae and invariance properties of the indexes used in experiments. For a comprehensive general review of linear indexes for measuring multivariate similarity, see Ramsay et al. (1984).

Linear Regression. A simple way to relate neural network representations is via linear regression. One can fit every feature in X as a linear combination of features from Y. A suitable summary statistic is the total fraction of variance explained by the fit:

$$R^2_{\text{LR}} = 1 - \frac{\min_B ||X - YB||_{\text{F}}^2}{||X||_{\text{F}}^2} = \frac{||Q_Y^{\text{T}}X||_{\text{F}}^2}{||X||_{\text{F}}^2}. \quad (5)$$

We are unaware of any application of linear regression to measuring similarity of neural network representations, although Romero et al. (2015) used a least squares loss between activations of two networks to encourage thin and deep "student" networks to learn functions similar to wide and shallow "teacher" networks.

Canonical Correlation Analysis (CCA). Canonical correlation finds bases for two matrices such that, when the original matrices are projected onto these bases, the correlation is maximized. For $1 \leq i \leq p_1$, the ith canonical correlation coefficient ρ_i is given by:

$$\begin{aligned} \rho_i = \max_{\mathbf{w}_X^i, \mathbf{w}_Y^i} & \text{corr}(X\mathbf{w}_X^i, Y\mathbf{w}_Y^i) \\ \text{subject to} \quad & \forall_{j<i} \; X\mathbf{w}_X^i \perp X\mathbf{w}_X^j \\ & \forall_{j<i} \; Y\mathbf{w}_Y^i \perp Y\mathbf{w}_Y^j. \end{aligned} \quad (6)$$

The vectors $\mathbf{w}_X^i \in \mathbb{R}^{p_1}$ and $\mathbf{w}_Y^i \in \mathbb{R}^{p_2}$ that maximize ρ_i are the canonical weights, which transform the original data into canonical variables $X\mathbf{w}_X^i$ and $Y\mathbf{w}_Y^i$. The constraints in (6) enforce orthogonality of the canonical variables.

For the purpose of this work, we consider two summary

statistics of the goodness of fit of CCA:

$$R^2_{\text{CCA}} = \frac{\sum_{i=1}^{p_1} \rho_i^2}{p_1} = \frac{||Q_Y^{\text{T}} Q_X||_{\text{F}}^2}{p_1} \tag{7}$$

$$\bar{\rho}_{\text{CCA}} = \frac{\sum_{i=1}^{p_1} \rho_i}{p_1} = \frac{||Q_Y^{\text{T}} Q_X||_*}{p_1}, \tag{8}$$

where $||\cdot||_*$ denotes the nuclear norm. The mean squared CCA correlation R^2_{CCA} is also known as *Yanai's GCD measure* (Ramsay et al., 1984), and several statistical packages report the sum of the squared canonical correlations $p_1 R^2_{\text{CCA}} = \sum_{i=1}^{p_1} \rho_i^2$ under the name *Pillai's trace* (SAS Institute, 2015; StataCorp, 2015). The mean CCA correlation $\bar{\rho}_{\text{CCA}}$ was previously used to measure similarity between neural network representations in Raghu et al. (2017).

SVCCA. CCA is sensitive to perturbation when the condition number of X or Y is large (Golub & Zha, 1995). To improve robustness, *singular vector CCA* (SVCCA) performs CCA on truncated singular value decompositions of X and Y (Raghu et al., 2017; Mroueh et al., 2015; Kuss & Graepel, 2003). As formulated in Raghu et al. (2017), SVCCA keeps enough principal components of the input matrices to explain a fixed proportion of the variance, and drops remaining components. Thus, it is invariant to invertible linear transformation only if the retained subspace does not change.

Projection-Weighted CCA. Morcos et al. (2018) propose a different strategy to reduce the sensitivity of CCA to perturbation, which they term "projection-weighted canonical correlation" (PWCCA):

$$\rho_{\text{PW}} = \frac{\sum_{i=1}^{c} \alpha_i \rho_i}{\sum_{i=1} \alpha_i} \qquad \alpha_i = \sum_j |\langle \mathbf{h}_i, \mathbf{x}_j \rangle|, \tag{9}$$

where $\mathbf{x}_j$ is the jth column of X, and $\mathbf{h}_i = X\mathbf{w}_X^i$ is the vector of canonical variables formed by projecting X to the ith canonical coordinate frame. As we show in Appendix C.3, PWCCA is closely related to linear regression, since:

$$R^2_{\text{LR}} = \frac{\sum_{i=1}^{c} \alpha_i' \rho_i^2}{\sum_{i=1} \alpha_i'} \qquad \alpha_i' = \sum_j \langle \mathbf{h}_i, \mathbf{x}_j \rangle^2. \tag{10}$$

Neuron Alignment Procedures. Other work has studied alignment between individual neurons, rather than alignment between subspaces. Li et al. (2015) examined correlation between the neurons in different neural networks, and attempt to find a bipartite match or semi-match that maximizes the sum of the correlations between the neurons, and then to measure the average correlations. Wang et al. (2018) proposed to search for subsets of neurons $\tilde{X} \subset X$ and $\tilde{Y} \subset Y$ such that, to within some tolerance, every neuron in $\tilde{X}$ can be represented by a linear combination of neurons from $\tilde{Y}$ and vice versa. They found that the maximum matching subsets are very small for intermediate layers.

Mutual Information. Among non-linear measures, one candidate is mutual information, which is invariant not only to invertible linear transformation, but to any invertible transformation. Li et al. (2015) previously used mutual information to measure neuronal alignment. In the context of comparing representations, we believe mutual information is not useful. Given any pair of representations produced by deterministic functions of the same input, mutual information between either and the input must be at least as large as mutual information between the representations. Moreover, in fully invertible neural networks (Dinh et al., 2017; Jacobsen et al., 2018), the mutual information between any two layers is equal to the entropy of the input.

5. Linear CKA versus CCA and Regression

Linear CKA is closely related to CCA and linear regression. If X and Y are centered, then Q_X and Q_Y are also centered, so:

$$R^2_{\text{CCA}} = \text{CKA}(Q_X Q_X^{\text{T}}, Q_Y Q_Y^{\text{T}}) \sqrt{\frac{p_2}{p_1}}. \tag{11}$$

When performing the linear regression fit of X with design matrix Y, $R^2_{\text{LR}} = ||Q_Y^{\text{T}} X||_F^2 / ||X||_F^2$, so:

$$R^2_{\text{LR}} = \text{CKA}(XX^{\text{T}}, Q_Y Q_Y^{\text{T}}) \frac{\sqrt{p_1} ||X^{\text{T}} X||_{\text{F}}}{||X||_{\text{F}}^2}. \tag{12}$$

When might we prefer linear CKA over CCA? One way to show the difference is to rewrite X and Y in terms of their singular value decompositions $X = U_X \Sigma_X V_X^{\text{T}}$, $Y = U_Y \Sigma_Y V_Y^{\text{T}}$. Let the ith eigenvector of XX^{T} (left-singular vector of X) be indexed as $\mathbf{u}_X^{\text{i}}$. Then R^2_{CCA} is:

$$R^2_{\text{CCA}} = ||U_Y^{\text{T}} U_X||_{\text{F}}^2 / p_1 = \sum_{i=1}^{p_1} \sum_{j=1}^{p_2} \langle \mathbf{u}_X^i, \mathbf{u}_Y^j \rangle^2 / p_1. \tag{13}$$

Let the ith eigenvalue of XX^{T} (squared singular value of X) be indexed as λ_X^i. Linear CKA can be written as:

$$\begin{aligned} \text{CKA}(XX^{\text{T}}, YY^{\text{T}}) &= \frac{||Y^{\text{T}} X||_{\text{F}}^2}{||X^{\text{T}} X||_{\text{F}} ||Y^{\text{T}} Y||_{\text{F}}} \\ &= \frac{\sum_{i=1}^{p_1} \sum_{j=1}^{p_2} \lambda_X^i \lambda_Y^j \langle \mathbf{u}_X^i, \mathbf{u}_Y^j \rangle^2}{\sqrt{\sum_{i=1}^{p_1} (\lambda_X^i)^2} \sqrt{\sum_{j=1}^{p_2} (\lambda_Y^j)^2}}. \end{aligned} \tag{14}$$

Linear CKA thus resembles CCA weighted by the eigenvalues of the corresponding eigenvectors, *i.e.* the amount of variance in X or Y that each explains. SVCCA (Raghu et al., 2017) and projection-weighted CCA (Morcos et al., 2018) were also motivated by the idea that eigenvectors that correspond to small eigenvalues are less important, but

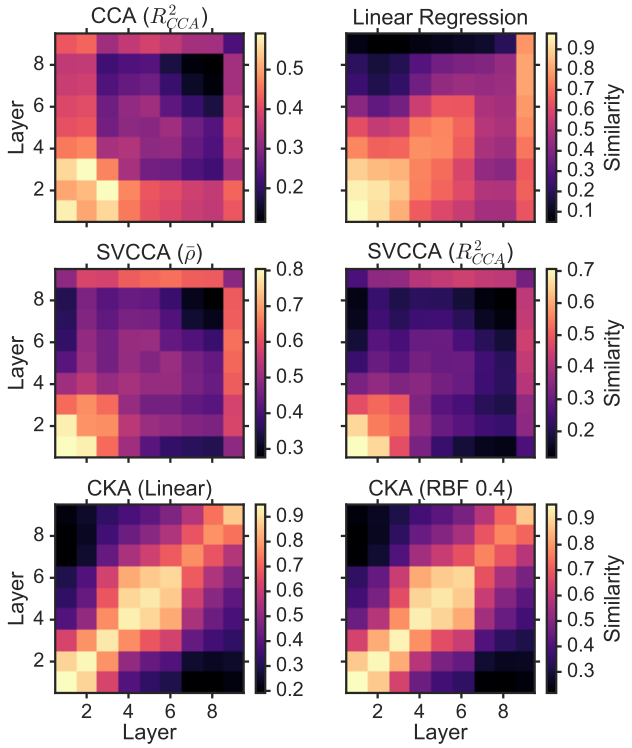

Figure 2. CKA reveals consistent relationships between layers of CNNs trained with different random initializations, whereas CCA, linear regression, and SVCCA do not. For linear regression, which is asymmetric, we plot R^2 for the fit of the layer on the x-axis with the layer on the y-axis. Results are averaged over 10 networks on the CIFAR-10 training set. See Table 2 for a numerical summary.

linear CKA incorporates this weighting symmetrically and can be computed without a matrix decomposition.

Comparison of (13) and (14) immediately suggests the possibility of alternative weightings of scalar products between eigenvectors. Indeed, as we show in Appendix D.1, the similarity index induced by "canonical ridge" regularized CCA (Vinod, 1976), when appropriately normalized, interpolates between R^2_{CCA}, linear regression, and linear CKA.

6. Results

6.1. A Sanity Check for Similarity Indexes

We propose a simple sanity check for similarity indexes: Given a pair of architecturally identical networks trained from different random initializations, for each layer in the first network, the most similar layer in the second network should be the architecturally corresponding layer. We train 10 networks and, for each layer of each network, we compute the accuracy with which we can find the corresponding layer in each of the other networks by maximum similarity. We then average the resulting accuracies. We compare CKA with CCA, SVCCA, PWCCA, and linear regression.

Index	Accuracy
CCA ($\bar{\rho}$)	1.4
CCA (R^2_{CCA})	10.6
SVCCA ($\bar{\rho}$)	9.9
SVCCA (R^2_{CCA})	15.1
PWCCA	11.1
Linear Reg.	45.4
Linear HSIC	22.2
CKA (Linear)	**99.3**
CKA (RBF 0.2)	80.6
CKA (RBF 0.4)	**99.1**
CKA (RBF 0.8)	**99.3**

Table 2. Accuracy of identifying corresponding layers based on maximum similarity for 10 architecturally identical 10-layer CNNs trained from different initializations, with logits layers excluded. For SVCCA, we used a truncation threshold of 0.99 as recommended in Raghu et al. (2017). For asymmetric indexes (PWCCA and linear regression) we symmetrized the similarity as $S + S^{\text{T}}$. CKA RBF kernel parameters reflect the fraction of the median Euclidean distance used as σ. Results not significantly different from the best result are bold-faced ($p < 0.05$, jackknife z-test).

We first investigate a simple VGG-like convolutional network based on All-CNN-C (Springenberg et al., 2015) (see Appendix E) on CIFAR-10. Figure 2 and Table 2 show that CKA passes our sanity check, but other methods perform substantially worse. For SVCCA, we experimented with a range of truncation thresholds, but no threshold revealed the layer structure (Appendix F.2); our results are consistent with those in Appendix E of Raghu et al. (2017).

We also investigate Transformer networks, where all layers are of equal width. In Appendix F.1, we show similarity between the 12 sublayers of the encoders of Transformer models (Vaswani et al., 2017) trained from different random initializations. All similarity indexes achieve non-trivial accuracy and thus pass the sanity check, although RBF CKA and R^2_{CCA} performed slightly better than other methods. However, we found that there are differences in feature scale between representations of feed-forward network and self-attention sublayers that CCA does not capture because it is invariant to non-isotropic scaling.

6.2. Using CKA to Understand Network Architectures

CKA can reveal pathology in neural networks representations. In Figure 3, we show CKA between layers of individual CNNs with different depths, where layers are repeated 2, 4, or 8 times. Doubling depth improved accuracy, but greater multipliers hurt accuracy. At 8x depth, CKA indicates that representations of more than half of the network are very similar to the last layer. We validated that these later layers do not refine the representation by training an ℓ^2-regularized logistic regression classifier on each layer of the network. Classification accuracy in shallower architectures progressively improves with depth, but for the 8x deeper